

Topic 3 — Cloud Monitoring, Kubernetes Scaling, AI-Driven Cost Optimization & Production Best Practices

Detailed Architecture, Design, Operations and Interview/Presentation Guide

Prepared: September 2026

How to use this guide

This document expands each presentation slide into a speaker-ready explanation. It is designed for architecture discussions, technical interviews, customer presentations and production design reviews.

1. Executive Overview

Build a production platform that observes cloud infrastructure and applications, scales Kubernetes based on demand, and continuously optimizes cost without compromising reliability, security or performance.

2. Cloud Monitoring Model

Monitor infrastructure, platform services, applications and business SLIs. Combine provider-native monitoring with application telemetry and OpenTelemetry where appropriate.

Organize dashboards around services and user journeys, not only individual resources.

3. Golden Signals & SLOs

Latency, traffic, errors and saturation provide a practical starting point. SLOs turn telemetry into an explicit reliability target.

Use error budgets to balance release velocity and operational risk.

4. Multi-Cloud Monitoring

Normalize common dimensions such as service, environment, region, account/project/subscription, owner and cost center. Keep provider-specific signals where they carry important operational meaning.

Centralize critical alerts while preserving local troubleshooting tools.

5. Kubernetes Observability

Monitor cluster capacity, node pressure, pod health, workload replicas, scheduling, container resources, API-server health and application SLOs.

Use labels and annotations consistently to connect workloads to owners, environments and services.

Architecture checkpoint

Ask: What is the scale, failure mode, security boundary, operational owner, recovery path and measurable success criterion for this design?

6. Kubernetes Scaling Concepts

Horizontal Pod Autoscaler adjusts replicas. Vertical Pod Autoscaler can recommend or adjust resource requests/limits depending on operating mode. Cluster Autoscaler changes node capacity. Event-driven autoscaling can respond to queue depth or external signals.

7. Scaling Decision Flow

Demand signal → metric validation → target calculation → replica change → scheduler placement → node scaling if needed → readiness verification → stabilization.

Avoid scaling based on noisy or lagging signals.

8. Resource Requests & Limits

Requests influence scheduling; limits constrain runtime usage. Poor sizing can cause waste, throttling, OOM kills or scheduling failures.

Use historical utilization and workload behavior to tune requests gradually.

9. AI-Driven Cost Optimization

AI can identify idle resources, oversized instances, underutilized clusters, storage inefficiencies and anomalous spending. It can forecast demand and recommend rightsizing or scheduling changes.

Recommendations should be evidence-based and bounded by reliability constraints.

10. Cost Attribution

Use tags/labels for application, team, environment, product and cost center. Allocate shared costs using transparent rules.

Track unit economics such as cost per transaction, customer, API request or workload hour.

Architecture checkpoint

Ask: What is the scale, failure mode, security boundary, operational owner, recovery path and measurable success criterion for this design?

11. Forecasting & Anomaly Detection

Time-series models can forecast expected spend or utilization. Anomaly detection can flag unexpected deviations.

A useful system combines statistical/ML detection with business context, seasonality, deployment events and known maintenance windows.

12. Optimization Guardrails

Never optimize cost blindly. Define minimum capacity, SLO thresholds, availability constraints, compliance restrictions, performance limits and change windows.

Test recommendations in non-production and use staged rollout.

13. Production Best Practices

Use infrastructure as code, immutable deployments where practical, centralized secrets, least privilege, automated testing, progressive delivery, backup/restore tests, disaster recovery exercises and continuous observability.

14. Reference Production Architecture

Cloud services + Kubernetes → OpenTelemetry/Cloud Monitoring → centralized metrics/logs/traces → event/alert pipeline → cost data + usage telemetry → optimization engine → recommendation/approval → IaC or automation → validation.

Feed outcomes back into the optimization model.

15. Executive KPIs & Example

Track availability, SLO attainment, MTTR, utilization, autoscaling accuracy, cost variance, cost per business unit, savings realized, recommendation acceptance and reliability impact.

Example: a variable-demand API uses HPA for pods, cluster autoscaling for nodes, workload-specific cost attribution and AI-assisted rightsizing. Changes are approved, staged and validated against latency/error SLOs.

Architecture checkpoint

Ask: What is the scale, failure mode, security boundary, operational owner, recovery path and measurable success criterion for this design?

Reference Areas

- Kubernetes documentation
- Prometheus documentation
- AWS Well-Architected Framework
- Google Cloud Architecture Framework
- Azure Well-Architected Framework

Note: Product capabilities and configuration details vary by cloud/provider version. Validate implementation details against the current official product documentation before production rollout.